

When the Write Lands but the Acknowledgement Is Lost: A Reproducible Memory-Retry Test in OpenClaw

Javad Taghia
AtMem.Ai Lab
<https://atmem.ai>

September 27, 2026

Abstract

A memory write can succeed even when its caller receives a timeout. Retrying then raises a question that recall accuracy alone cannot answer: does the agent read one complete original record, a duplicate, or conflicting state? We present a small, reproducible fault-injection study of the AtMem `memory_remember` integration with OpenClaw. A transport proxy observes a successful backend response, withholds it, and kills the backend process; the caller receives a real timeout and retries. The final qualification uses five isolated bridge trials and three real OpenClaw gateway trials, plus one gateway no-fault control. In every faulted qualification trial, retries return the original record identifier with status `already_stored`; the final store contains one active record with the complete original fact and a valid audit chain. Gateway trials use deterministic local model responses, not paid inference or an autonomous learned retry policy. We retain unsuccessful CLI setup attempts separately. These results establish recovery of one exact synthetic fact after a completed write and lost response in the tested configuration. They do not establish transaction-interruption safety, paraphrase deduplication, or a general exactly-once guarantee.

Keywords: persistent memory, OpenClaw, acknowledgement loss, tool timeout, retry, fault injection, reproducibility

1 The question

Consider an agent instructed to remember a preference. Its tool sends a write to a persistent memory service. The service commits the record, but the reply never reaches the agent. The agent sees a timeout, retries, and reads back the memory. Returning the right words is necessary but insufficient: the retry could have created two records, changed the source linkage, or overwritten metadata. A useful test therefore inspects both the agent-visible tool result and the persisted state.

This note follows the broader integrity and continuity evaluation in *Beyond Recall Accuracy* [1]. Its question is narrower: *what state does an OpenClaw agent read after a successful memory write whose acknowledgement was lost?* The experiment tests an actual memory write, rather than an external document publication or retail exchange.

Two different failure windows. A partially completed transaction and a completed write with a lost response are distinct. Here the fault is injected only after the backend has produced a successful response containing a new record. This is a post-write failure window. Killing the backend before the response is delivered does not test every point inside its storage transaction or derived-index updates.

2 System and test boundary

The tested path is OpenClaw’s registered `memory_remember` tool, the production TypeScript AtMem bridge, and the real Python MCP backend with a file-backed SQLite memory store. Only the transport proxy and test driver are experimental. Product write, duplicate handling, record read, and audit-verification implementations are unchanged.

Qualification uses a clean exported source snapshot at commit `da046dc59fd41fdd77ed407d8f196cfc434b478c`. The bridge package declares version 2.3.7. The installed OpenClaw runtime reports 2026.9.1 (`ad6fe23`). This is a source-snapshot qualification, not certification of a newly released wheel. Runtime versions and source hashes are recorded in the accompanying artifact manifest.

Each trial starts with a new directory, memory database, and synthetic subject. Gateway trials additionally use an isolated OpenClaw state directory, workspace, plugin installation, loopback model endpoint, and gateway. They do not modify the user’s normal agent memory or send messages to external channels. No paid model calls are made.

2.1 Two execution levels

Bridge trials register the actual bridge plugin against a minimal OpenClaw API harness. The harness supplies a

synthetic authenticated-message binding and invokes the registered tool. It retries first with the original tool-call identifier and then with a new identifier. These trials directly inspect the complete record before and after retry.

Gateway trials run the installed OpenClaw gateway and ordinary agent command. A local OpenAI-compatible endpoint supplies a fixed sequence: write, retry with a new tool-call identifier, read the returned record, finish. OpenClaw executes the real tools and passes their actual results back to the endpoint. Its normal lifecycle hooks stage the source message. The endpoint chooses the read identifier from the observed tool result, not from the evaluator’s private copy of the withheld response.

This second level establishes host integration, not language-model reasoning quality. The retry policy is scripted. A learned model might retry differently, change the wording, or decline to retry; those behaviors are outside this study.

3 Protocol and assertions

The synthetic user request is to remember a preference for cobalt blue notebooks. The proposed memory is exactly:

User prefers cobalt blue notebooks.

The replaceable slot is `notebook_preference`. Source type is `user_message`. Each trial follows this sequence:

1. Stage the source message through the normal host hook, or explicitly through the real staging tool in the bridge harness.
2. Invoke `memory_remember` with the synthetic fact and slot.
3. The proxy waits for a successful MCP reply containing a newly created record. It saves the request and response as evaluator evidence, never forwards that reply, and sends `SIGKILL` to the backend process.
4. Keep the transport open so the real bridge request deadline expires after 2,000 ms. The caller receives a timeout rather than a success response.
5. Reconnect to a fresh backend process using the same database and retry the identical fact. Bridge trials exercise both unchanged and new call IDs; gateway trials exercise a new call ID.
6. Read the record through `memory_get`, enumerate all records including inactive ones, and verify retained audit evidence.

The gateway endpoint waits 600 ms before returning each scripted response, allowing the bridge’s 250 ms idle-close interval to dispose of the faulted transport. Bridge trials explicitly stop their registered service before reconnecting. Immediate retry on a still-open faulted connection is not qualified by this schedule. The operating system and gateway remain alive; the process killed is the memory backend.

Success criteria. A faulted trial must show a committed record in the withheld reply, an actual caller-visible timeout, the original identifier in the successful retry, `stored=true`, `status=already_stored`, one final active record, the exact original fact on read-back, one `memory.record_created` audit event, and a valid audit chain. Bridge qualification additionally requires equality of the entire record list before and after both retries. Gateway qualification additionally checks SQLite integrity and foreign-key consistency. Normal host termination alone is not a pass.

4 Results

Table 1 separates execution levels and the control. Counts describe repeated deterministic engineering trials, not sampled production failure probabilities. All final qualification trials use the same fact and fault boundary, with independent temporary stores.

Table 1: Final qualification. Each successful run ends with one record. The two retries in a bridge trial are sequential, not independent trials.

Condition	Runs	Passes
Bridge: lost response, two retry IDs	5	5
OpenClaw gateway: lost response	3	3
OpenClaw gateway: no-fault control	1	1

4.1 What the agent reads back

The lost-response gateway sequence exposes a timeout on the first call. The retry returns the original record ID, `stored=true`, and `already_stored`. The subsequent `memory_get` returns “User prefers cobalt blue notebooks.” Its provenance identifies `typed_session_handoff` and a host-asserted semantic interpretation.

Inspection of the final stored record shows the original source turn, original episode linkage, `trusted_user` trust tier, active status, the same fact key, generation zero, and no superseding record. Including inactive records does not reveal a hidden duplicate. In the bridge trials, the complete record dictionaries before and after both retries are identical, including creation timestamp and metadata.

The no-fault gateway control receives success on the first write and `already_stored` on its deliberate repeat. This confirms that the scripted retry and read-back also work without the injected failure.

4.2 Record identity and retry events

The audit trail retains the original creation and duplicate-recognition events. There is one record-creation event per trial, but retries are still invocations and are still observable. Bridge trials record two semantic duplicate

events; gateway trials record one. We therefore describe the result as *one preserved canonical record under exact-content retry*, rather than claiming that every side effect occurred exactly once.

4.3 Development and setup outcomes

Before final qualification, six bridge development trials passed. Two embedded OpenClaw CLI runs failed before reaching the fault boundary: no current typed user-message binding was available, so the write was rejected and the store remained empty. These are failed setup attempts, not successful crash tests or memory corruption. Changing to an isolated gateway startup allowed source staging and the subsequent development gateway trial passed.

The final qualification was then run from the clean source snapshot. The artifact index retains the development outcomes and marks their different provenance. The two failed embedded runs remain an integration limitation; this experiment does not establish their root cause or claim a product fix.

5 Interpretation and limitations

The direct answer to the motivating question is: *in these tests, the agent reads the same complete original memory, and its retry is reported as already stored*. The result is stronger than checking that recall returns the expected phrase because it also checks identity, record multiplicity, metadata preservation, and audit evidence.

It is nevertheless narrow. A new call identifier still used identical fact bytes and the same slot and source binding. This demonstrates the observed duplicate handling; it does not establish an API-wide idempotency-key contract. Changing the fact, paraphrasing it, changing the source, or racing concurrent writers could produce different behavior. No baseline or competing memory system is evaluated, and no superiority claim follows.

The backend was killed after emitting a successful response, not during individual SQL statements. No whole-machine crash, sudden power loss, torn storage write, filesystem failure, or distributed replica failure was injected. SQLite integrity checks and valid audit hashes do not prove those untested properties. Nor do they independently establish the truth of a user’s fact.

The host run used an explicitly enabled legacy direct-memory tool profile, not every managed control-plane configuration or arbitrary OpenClaw tool. The deterministic endpoint tests actual tool dispatch and result handling, but not autonomous recovery planning. The deliberately paced retry follows transport reconnection; alternate retry timing remains untested.

6 Reproduction and artifacts

The public supplement contains the LaTeX source, PDF, probe programs, clean-source snapshot, per-run JSON evidence, a machine-readable summary, runtime and source hashes, and reproduction instructions. It is published alongside the earlier paper in the Hugging Face evidence dataset. The source snapshot and original backend response make the fault boundary inspectable. The saved response is evaluator-only evidence; the agent receives the timeout and must retry to obtain its record identifier.

No existing results or earlier paper are replaced. The supplement is a functional qualification note and should not be interpreted as a new leaderboard score or a general reliability rate.

References

- [1] Javad Taghia. Beyond Recall Accuracy: Evaluating Integrity and Crash Continuity in Persistent Memory for Tool-Using Language Agents. AtMem.Ai Lab technical manuscript, September 26, 2026. Manuscript and accompanying evidence on Hugging Face.